Operating Systems

Project Checkpoint 1

112070002　張博安

Table of Contents

---

1 Screenshots for Compilation

```
boanz@BENSONCH /cygdrive/d/OS/ppc1
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym *.asm *.lk
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
make: *** [Makefile:25: clean] Error 1

boanz@BENSONCH /cygdrive/d/OS/ppc1
$ make
sdcc -c  testcoop.c
testcoop.c:49: warning 158: overflow in implicit constant conversion
sdcc -c  cooperative.c
cooperative.c:180: warning 85: in function ThreadCreate unreferenced function ar
gument : 'fp'
cooperative.c:237: warning 158: overflow in implicit constant conversion
sdcc  -o testcoop.hex testcoop.rel cooperative.rel
```

Figure 1: Compilation result of `make clean` and `make`

The `make clean` command produced errors because some intermediate files (`.ihx`, `.lnk`) did not exist yet on a fresh build, which is expected behavior. The `make` command successfully compiled both `testcoop.c` and `cooperative.c` with three non-fatal warnings. First, `testcoop.c` line 49 produces a warning for implicit conversion of `-6` to an unsigned register `TH1`, which is the standard 8051 convention for baud rate configuration. Second, `cooperative.c` line 180 reports that the function argument `fp` in `ThreadCreate()` appears unreferenced to the C compiler, because `fp` is accessed through `DPTR` in inline assembly rather than directly in C. Third, `cooperative.c` line 237 produces a warning for implicit conversion of `-1` to `char` in `ThreadExit()`. No errors were produced and `testcoop.hex` was successfully generated.

2 Screenshots and Explanation

2.1 ThreadCreate

There are two `ThreadCreate` calls in total. The first one is in the `Bootstrap` function, which creates a thread for `main`. The second one is in the `main` function, which creates a thread for the `Producer`. The `main` function then runs the `Consumer` code directly, so the `Consumer` does not need an extra `ThreadCreate` call.

```
      Value  Global                              Global Defined In Module
      -----  --------------------------------   ------------------------
C:     0000004F  _Producer                         testcoop
C:     00000076  _Consumer                         testcoop
C:     0000009C  _main                             testcoop
C:     000000A8  __sdcc_gsinit_startup             testcoop
C:     000000AC  __mcs51_genRAMCLEAR               testcoop
C:     000000AD  __mcs51_genXINIT                  testcoop
C:     000000AE  __mcs51_genXRAMCLEAR              testcoop
C:     000000AF  _Bootstrap                        cooperative
C:     000000CD  _ThreadCreate                     cooperative
C:     00000151  _ThreadYield                      cooperative
C:     000001AA  _ThreadExit                       cooperative
```

Figure 2: Function address list

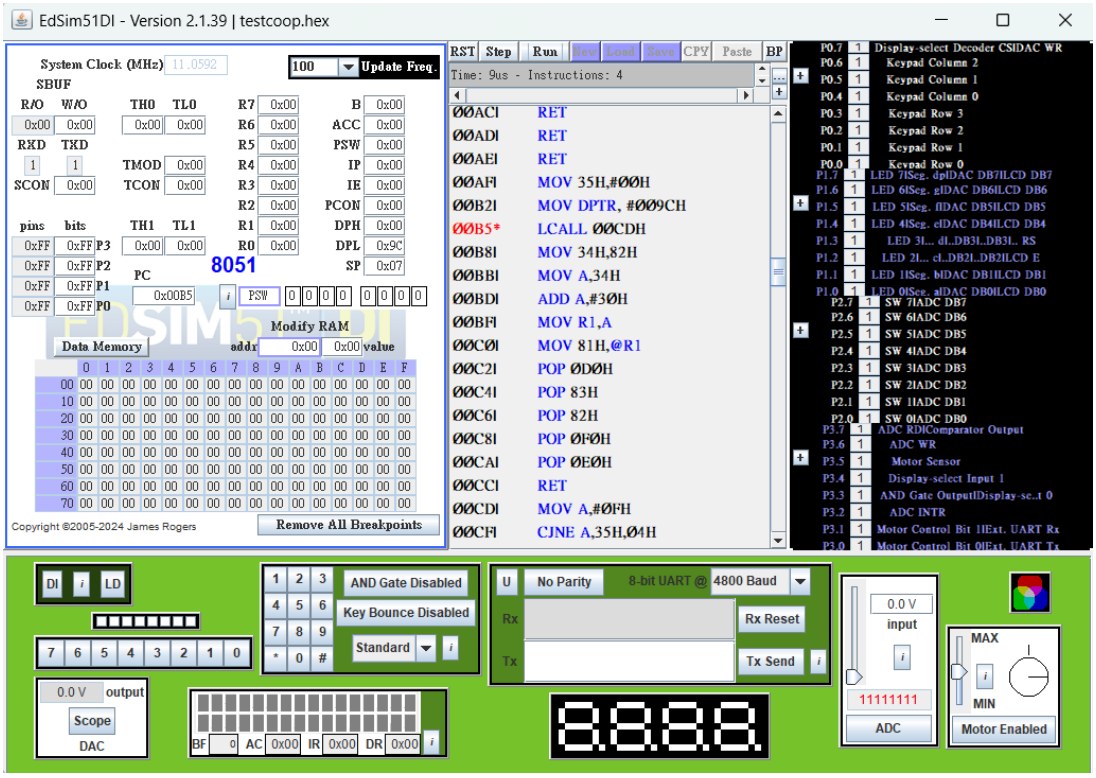

Figure 3: Screenshot before first `ThreadCreate` function call

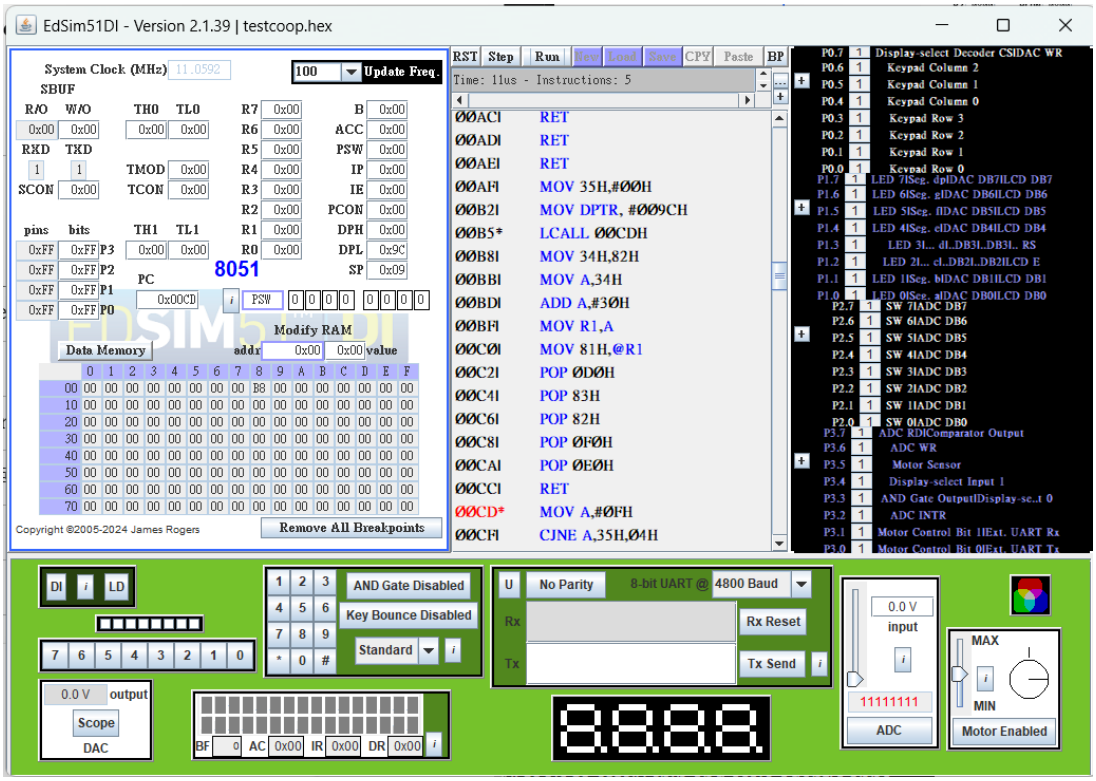

Figure 4: Screenshot after calling `ThreadCreate` function

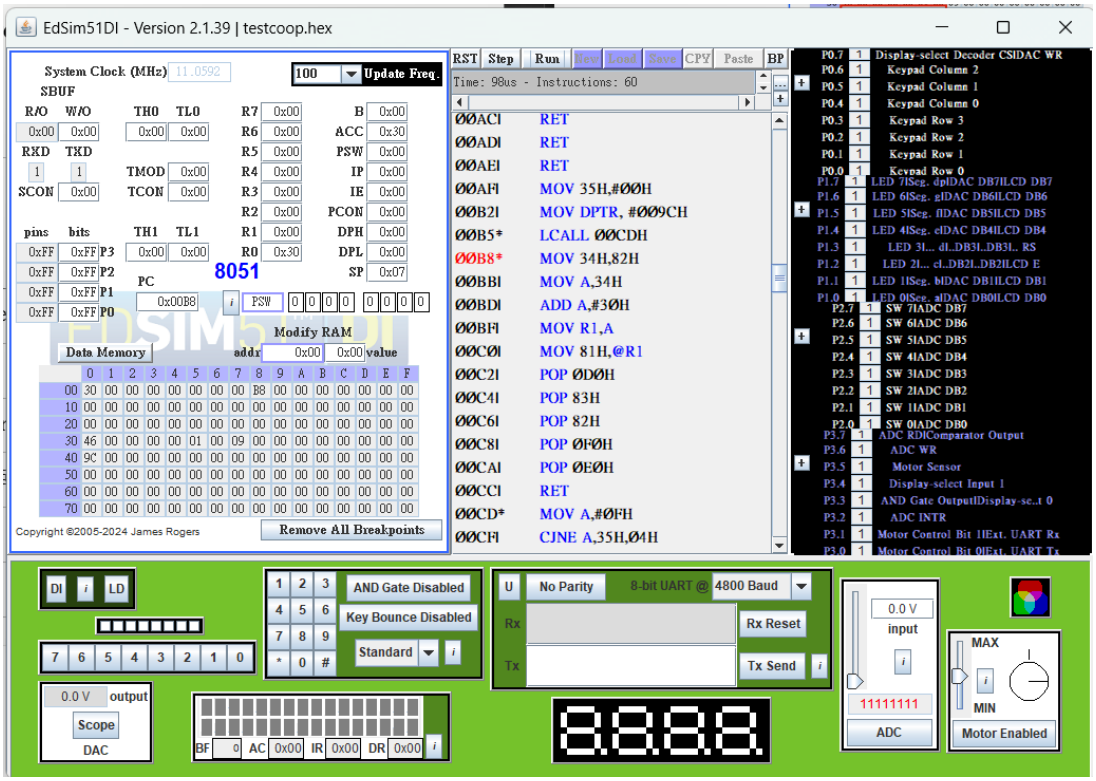

Figure 5: Screenshot after returning from first `ThreadCreate` call

First call — before (PC = 0x00B5): Before the first `ThreadCreate` call, the stack pointer SP = 0x07, which is the default value set by the 8051 hardware on reset. The thread bitmask (address 0x35) is 0x00, indicating that no thread has been created yet. `DPTR` holds 0x009C, which is the address of `main` and will be passed as the function pointer argument.

When the `LCALL 0x00CD` instruction is executed, the return address 0x00B8 is pushed onto the stack: the low byte 0xB8 is pushed to address 0x08, the high byte 0x00 is pushed to address 0x09, and SP is updated to 0x09.

First call — inside ThreadCreate (PC = 0x00CD): Inside `ThreadCreate`, the function first scans the bitmask for an unused thread. Since no thread exists, Thread 0 is assigned to `main` and the bitmask is updated to 0x01. It then computes the new stack start location ($\texttt{0x3F} + 0 \times \texttt{0x10} = \texttt{0x3F}$), saves the current SP to a temporary, and sets SP to 0x3F. The return address (DPL = 0x9C, DPH = 0x00) is pushed first, followed by ACC, B, DPL, DPH all initialized to 0. Finally PSW = ($0 \ll 3$) = 0x00 is pushed, selecting Register Bank 0 for Thread 0.

First call — after returning (PC = 0x00B8): After `ThreadCreate(main)` returns, SP is restored to 0x07. The new stack for Thread 0 now holds: address 0x40 = 0x9C (low byte of `main`), address 0x41 = 0x00 (high byte of `main`), and addresses 0x42–0x46 = 0x00 (the initialized ACC, B, DPL, DPH, and PSW). The saved stack pointer `savedSP[0]` (address 0x30) is set to 0x46. This layout lets `RESTORESTATE` immediately pop `main`'s context so it can start running.

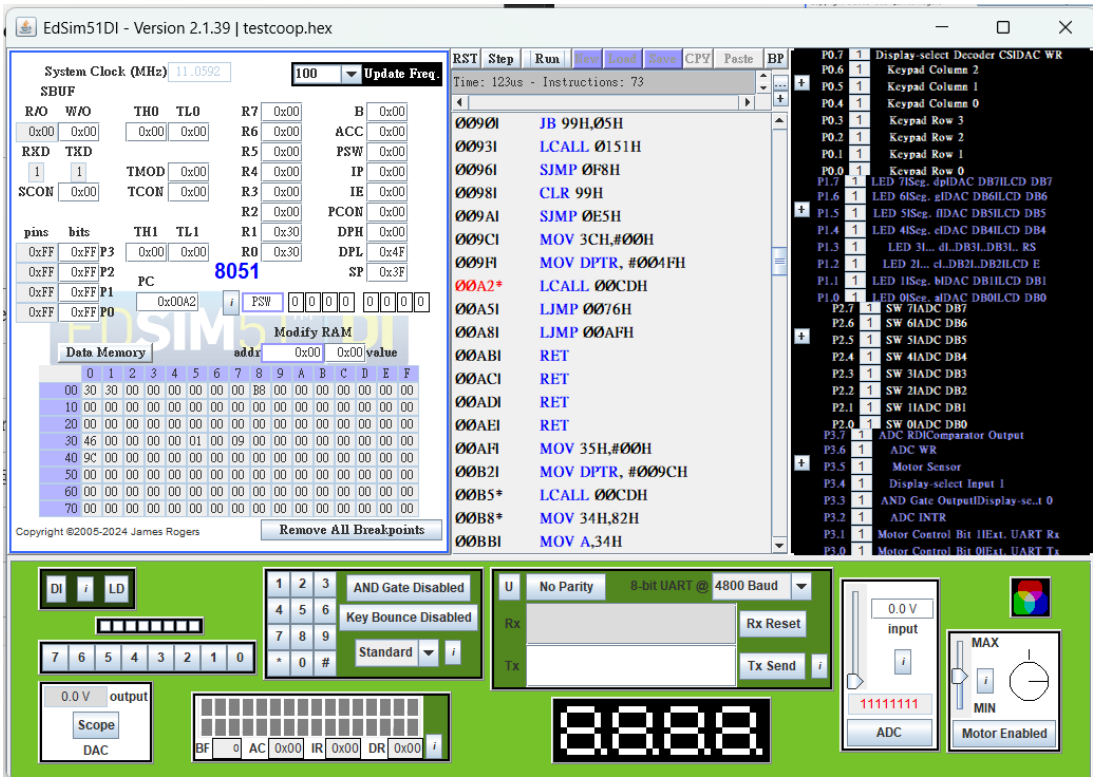

Figure 6: Screenshot before second `ThreadCreate` function call

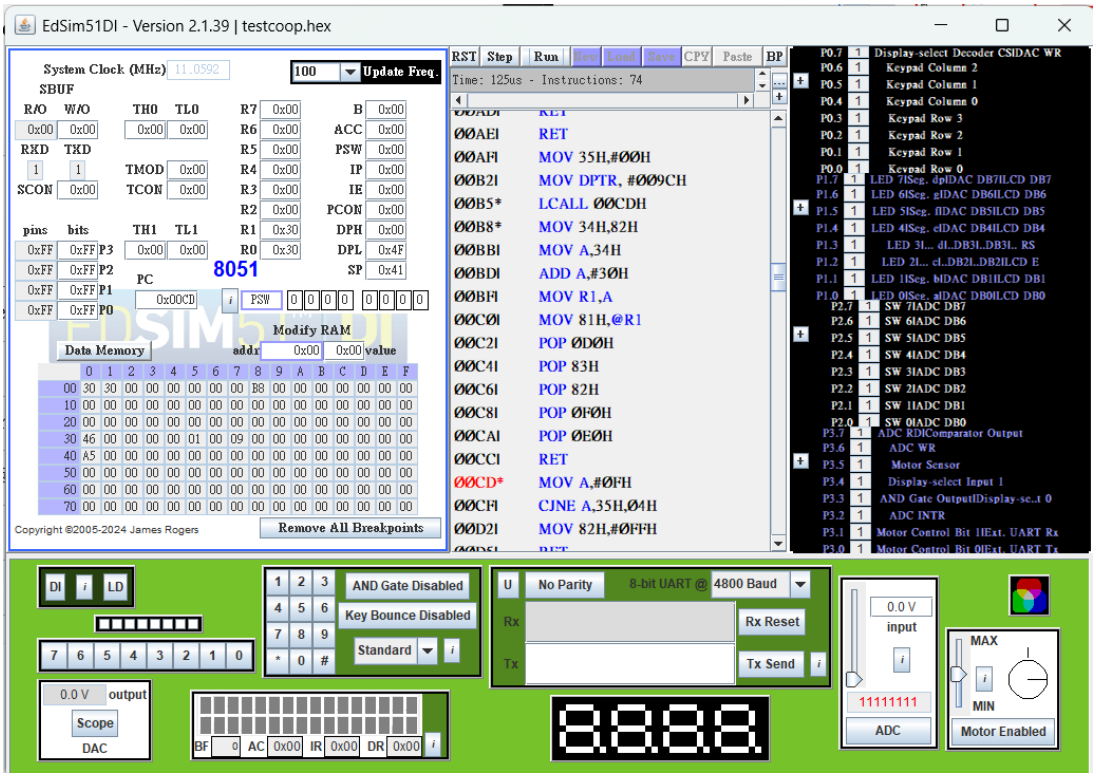

Figure 7: Screenshot after calling the second `ThreadCreate`

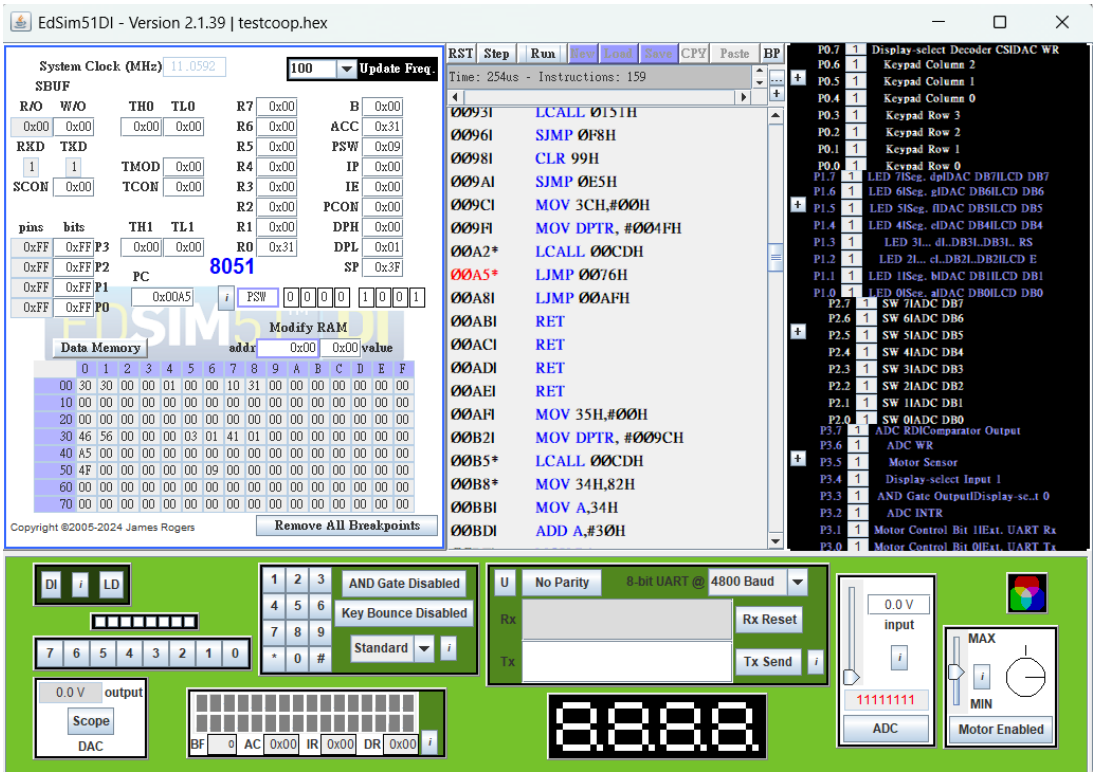

Figure 8: Screenshot after returning from second `ThreadCreate` call

Second call — before (PC = 0x00A2): Before the second `ThreadCreate` call, SP = 0x3F and the bitmask = 0x01, indicating Thread 0 (`main`) already exists. `DPTR` holds 0x004F, the address of `Producer`.

Second call — after returning (PC = 0x00A5): After `ThreadCreate(Producer)` returns, Thread 1 is assigned to `Producer` since Thread 0 is taken. The new stack for Thread 1 holds: address 0x50 = 0x4F (low byte of `Producer`), address 0x51 = 0x00 (high byte), addresses 0x52–0x55 = 0x00, and address 0x56 = 0x09. Here PSW = $(1 \ll 3)$ = 0x08 selects Register Bank 1, and the lowest bit is the parity bit, giving the stored value 0x09. The saved stack pointer `savedSP[1]` (address 0x31) is set to 0x56.

2.2 Producer

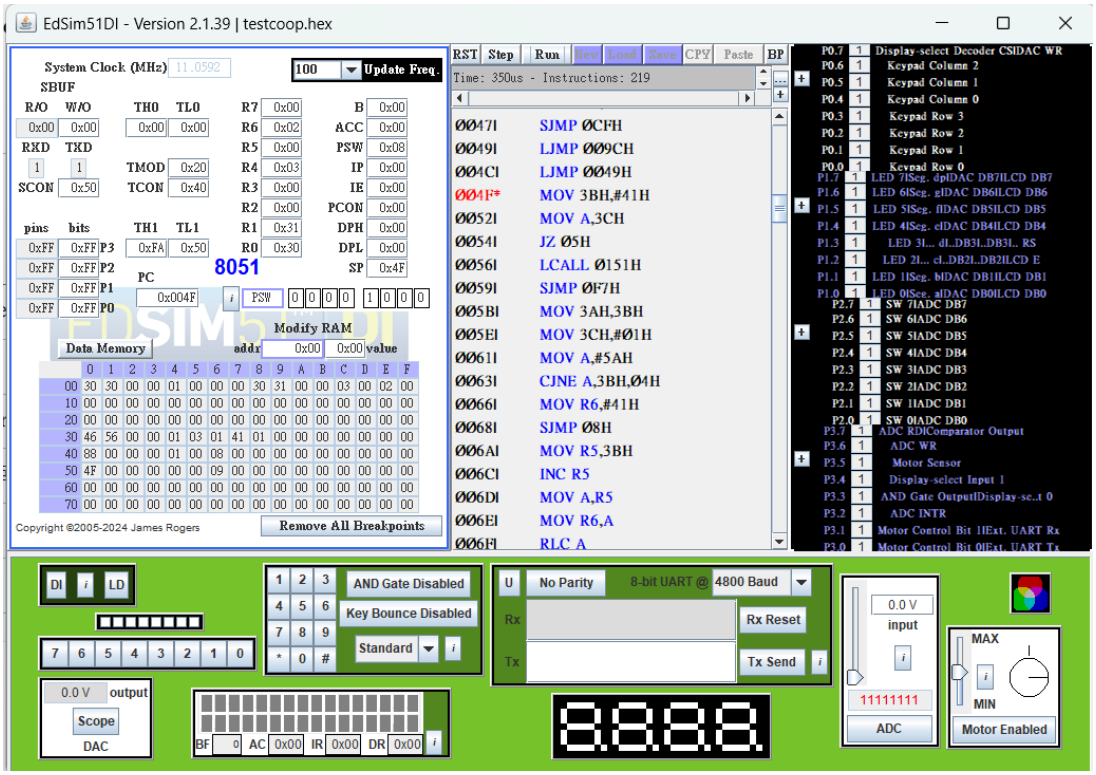

Figure 9: Screenshot for first `Producer` call

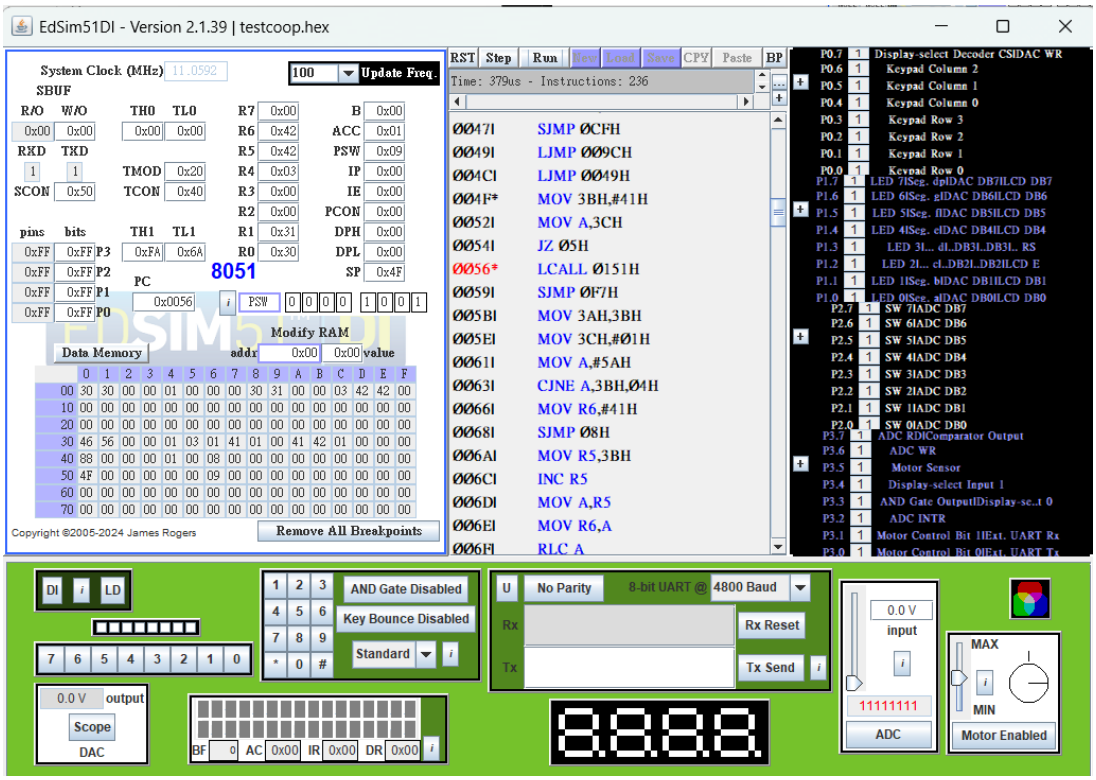

Figure 10: Screenshot after finishing first `Producer` call

Producer is running (PC = 0x004F): We can confirm the Producer is running from two pieces of evidence. First, PC = 0x004F matches the address of `_Producer` in the map file. Second, PSW = 0x08, meaning RS1:RS0 = 01, which selects Register Bank 1. Since `Producer` was created as Thread 1, and Thread 1 uses Register Bank 1, this confirms the Producer thread is executing. The variable `currentThread` (address 0x34) = 0x01 and the bitmask (address 0x35) = 0x03, confirming both Thread 0 and Thread 1 are active.

After the first Producer round (PC = 0x0056): In the first round, the Producer sets `currentChar` (address 0x3B) to 'A' (0x41) and enters its infinite loop. Since the buffer is empty, it writes `currentChar` into the buffer (address 0x3A), sets `bufferFull` (address 0x3C) to 1, and increments `currentChar` to 'B' (0x42). After the first round, the data memory shows: buffer (0x3A) = 0x41 ('A'), `currentChar` (0x3B) = 0x42 ('B'), and `bufferFull` (0x3C) = 0x01, confirming the Producer wrote the first character correctly. In later rounds, if the buffer is still full, the Producer yields immediately until the Consumer empties it.

2.3 Consumer

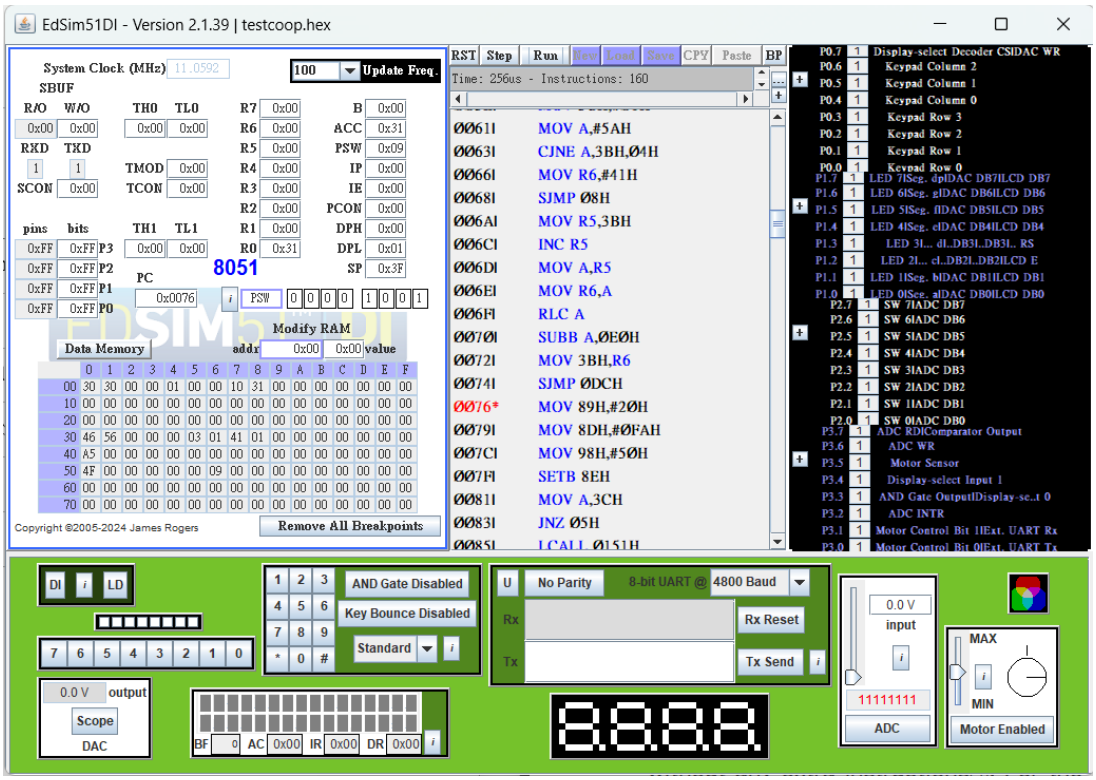

Figure 11: Screenshot for first `Consumer` call

Consumer is running (PC = 0x0076): We can confirm the Consumer is running from two pieces of evidence. First, PC = 0x0076 matches the address of `_Consumer` in the map file. Second, PSW = 0x09; the RS1:RS0 bits are 00, which selects Register Bank 0 (the lowest bit is the parity bit). Since the Consumer is called directly by `main`, which runs as Thread 0, and Thread 0 uses Register Bank 0, this confirms the Consumer is executing within Thread 0.

The Consumer first initializes the serial port for polling (TMOD, TH1, SCON, TR1). Because `main` runs the Consumer before the Producer has a chance to run, the buffer is empty in the first round, so the Consumer yields immediately to let the Producer produce a character. In subsequent rounds, once the buffer is full, the Consumer writes the buffer's character to SBUF, clears `bufferFull` to 0, waits (by yielding) until the TI flag indicates the transmission is complete, and then clears TI.

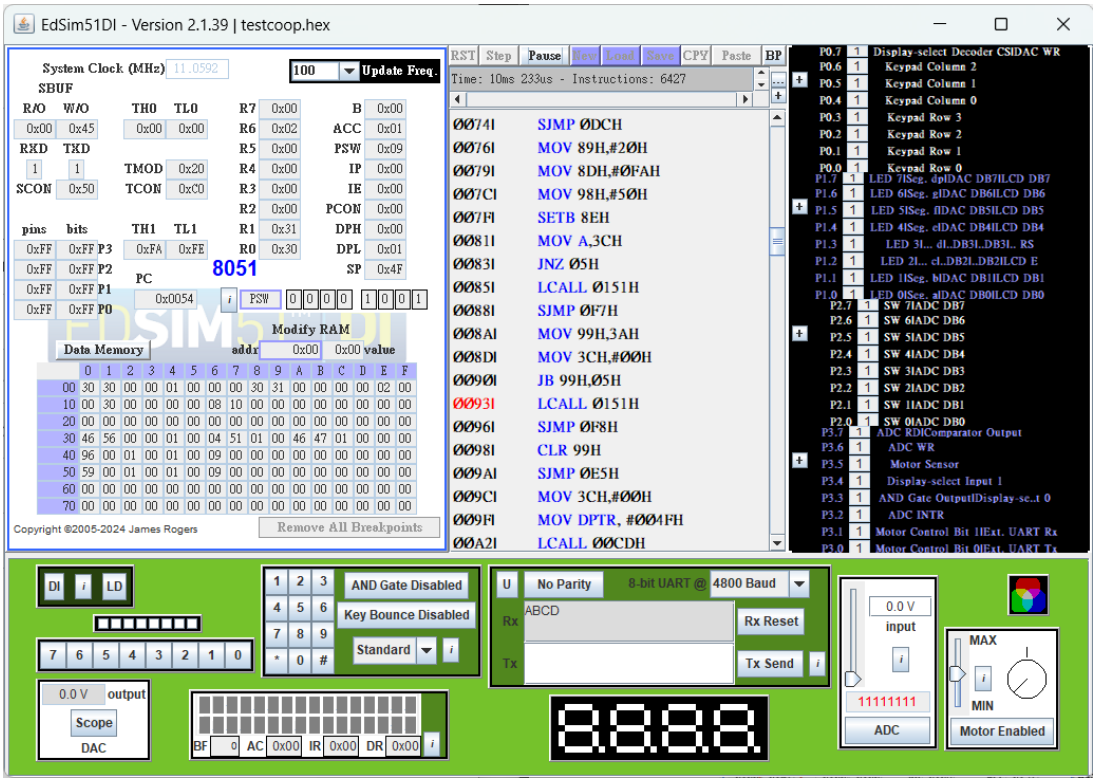

Figure 12: Screenshot for running 6427 instructions

Running for 6427 instructions: After running for a while, the Rx window shows the characters "ABCD", confirming that the producer-consumer cooperation works correctly: the `Producer` generates characters in sequence and the `Consumer` transmits them one by one through the serial port.